

Tendências de Motores de Jogos

Semana 11

Navigation, Behavior Tree, Blackboard e Combate Simples — Encerramento do Módulo 3

Disciplina: Tendências de Motores de Jogos (IN46A)

Instituição: IFMS — Campus Dourados

Curso: Tecnologia em Jogos Digitais

Módulo 3 — Resolver Problemas

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Objetivos da Semana

OBJETIVOS

- Compreender Navigation/NavMesh como base universal de deslocamento autônomo de agentes
- Compreender Behavior Tree como estrutura de decisão e Blackboard como memória compartilhada do agente
- Implementar BP_Enemy com patrulha e perseguição, reutilizando o HealthComponent já existente
- Implementar um combate simples: BP_Player ataca BP_Enemy via Trace/Overlap chamando ApplyDamage
- Propor, com autonomia própria, um comportamento autônomo adicional e apresentar o Vertical Slice do Módulo 3 em Showcase

Como a semana está organizada



Encontro 1

Navigation e NavMesh — `BP_Enemy` se desloca até um ponto



Encontro 2

Behavior Tree e Blackboard, combate simples (Trace/Overlap + `ApplyDamage`) — desafio de comportamento autônomo, Playtest e Showcase de encerramento do Módulo 3

Tendências de Motores de Jogos

ENCONTRO 1

Navigation e NavMesh

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Tendências de Motores de Jogos



Como um NPC sabe onde pode andar sem atravessar uma parede?

Pense na diferença entre a geometria que o jogador vê e colide, e a geometria que um agente autônomo consulta para planejar um caminho.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Geometria visual × geometria de navegação

- A geometria que o jogador vê e colide não é a mesma que um agente autônomo consulta para planejar caminho
- Um NPC precisa de uma malha navegável, calculada previamente a partir do nível
- Toda engine madura separa essas duas camadas: visual/colisão e navegação
- Este é o primeiro sistema do semestre dedicado exclusivamente a um agente não-jogador

DICA

Até aqui, todo deslocamento ensinado pertencia ao `BP_Player` — hoje o deslocamento passa a ser também autônomo, decidido por um agente.

NavMesh e AIController

- Nav Mesh Bounds Volume delimita a área onde a malha de navegação é gerada
- NavMesh é gerado automaticamente a partir da geometria marcada como estática dentro do volume
- AIController solicita deslocamento até qualquer ponto navegável da malha
- BP_Enemy (novo) é o primeiro Actor do projeto controlado por um AIController, não pelo jogador

Navigation: Unreal × Unity

Unreal

NavMesh gerado em tempo real dentro do

`Nav Mesh Bounds Volume`; `AIController`
solicita deslocamento sobre a malha

Unity

NavMesh gerado via bake explícito sobre a
geometria estática; `NavMeshAgent` solicita
deslocamento com `SetDestination`

Da geometria do nível ao deslocamento autônomo



Demonstração: NavMesh e primeiro deslocamento

O professor configura um `Nav Mesh Bounds Volume` cobrindo a área jogável, gera o NavMesh resultante, e cria um `BP_Enemy` com `AIController` capaz de se deslocar até um ponto fixo usando `Move To / Simple Move to Location`.

Resultado esperado: o `BP_Enemy` se move de um ponto a outro sem atravessar paredes ou obstáculos do nível.

Imagem sugerida

Objetivo: mostrar a malha de navegação gerada sobre o nível, reforçando a separação entre geometria visual e geometria navegável.

Enquadramento: captura de tela do editor da Unreal Engine com a visualização de NavMesh ativada sobre um trecho do Dungeon Kit.

Elementos presentes: área navegável destacada em verde translúcido sobre o piso do nível; contorno das paredes sem cobertura de NavMesh.

Destaque visual: o limite entre a área verde (navegável) e a geometria sem cobertura (paredes, obstáculos).

Legenda sugerida: "A malha de navegação existe apenas onde um agente pode efetivamente andar."

Boas práticas

✓ BOAS PRÁTICAS

- Marcar corretamente a geometria estática do nível antes de gerar o NavMesh
- Delimitar o `Nav Mesh Bounds Volume` cobrindo toda a área jogável, sem sobras
- Nomear e organizar `BP_Enemy` na subpasta `Blueprints/Characters/`, conforme `PROJECT_ARCHITECTURE.md`

Laboratório do Encontro 1

Configurar o NavMesh do próprio nível e criar `BP_Enemy` com `AIController`, validando o deslocamento até um ponto definido, sem alterar a lógica de nenhum sistema anterior.

OBJETIVOS

Critério de sucesso: NavMesh gerado corretamente sobre o nível e `BP_Enemy` deslocando-se até o ponto definido, sem atravessar paredes ou obstáculos.

Fechando o Encontro 1

- NavMesh separa geometria visual de geometria navegável; `AIController` consulta a malha para planejar deslocamento
- `BP_Enemy` de cada grupo deslocando-se corretamente até um ponto do nível
- Próximo encontro: dar a esse deslocamento uma estrutura de decisão — patrulhar ou perseguir

Tendências de Motores de Jogos

ENCONTRO 2

Behavior Tree e Blackboard

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Tendências de Motores de Jogos



Quem deve guardar "para onde o inimigo está indo": a árvore de decisão, ou outra coisa?

Pense no que acontece se a estrutura de decisão também tiver que guardar cada dado que usa.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Decisão e memória: dois papéis separados

- Uma Behavior Tree é uma estrutura hierárquica avaliada continuamente durante o jogo
- Nós de seleção escolhem entre ramos alternativos (patrulhar ou perseguir); nós de sequência executam passos em ordem
- Nenhum nó guarda por si só o estado do agente — essa responsabilidade é de outro artefato
- Um Blackboard é uma memória de chave-valor compartilhada, lida e escrita pela Behavior Tree a cada avaliação

DICA

O mesmo princípio de `BPI_Interactive /Event Dispatchers` (Semana 5): uma estrutura de controle não deve guardar o dado que outra parte do sistema também precisa consultar.

Patrulha e perseguição em BP_Enemy

- A Behavior Tree alterna entre dois ramos: patrulhar entre pontos e perseguir o jogador
- O Blackboard guarda chaves como `PontoDePatrulhaAtual` e `AlvoDetectado`
- A transição para perseguição ocorre quando `AlvoDetectado` é preenchido por uma verificação de distância ou visão
- `BP_Enemy` reutiliza o `HealthComponent` já construído na Semana 8, sem duplicar lógica de vida/dano

Behavior Tree e Blackboard: Unreal × Unity

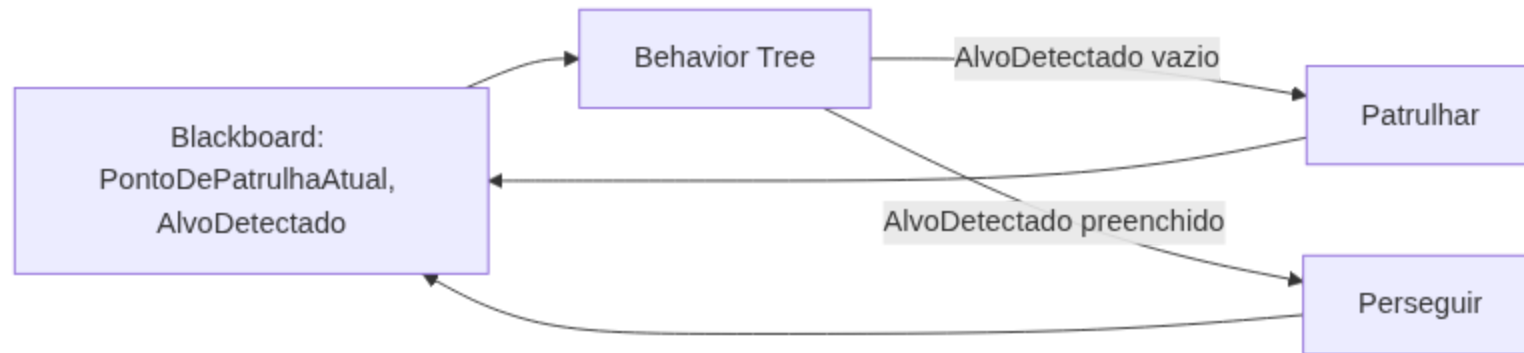
Unreal

Behavior Tree e Blackboard nativos, integrados ao editor desde o início

Unity

Sem equivalente nativo de mesmo nível; depende de packages de terceiros (Behavior Designer, NodeCanvas) para estrutura de decisão hierárquica, com memória compartilhada resolvida por variáveis do próprio asset ou um Dictionary

Behavior Tree e Blackboard em conjunto



Demonstração: patrulha e perseguição

O professor constrói uma Behavior Tree com um nó seletor entre dois ramos — patrulhar entre pontos definidos no Blackboard e perseguir o jogador quando a chave `AlvoDetectado` é preenchida por uma verificação de distância ou visão.

Resultado esperado: a turma vê o `BP_Enemy` alternando corretamente entre patrulha e perseguição, antes de propor o próprio comportamento adicional.

Imagem sugerida

Objetivo: mostrar a estrutura visual de uma Behavior Tree simples, reforçando a separação entre nós de decisão e o Blackboard que eles consultam.

Enquadramento: captura de tela do editor de Behavior Tree da Unreal Engine, com a janela de Blackboard visível ao lado.

*Elementos presentes: um nó Selector ligando os ramos "Patrulhar" e "Perseguir"; painel de Blackboard com as chaves **PontoDePatrulhaAtual** e **AlvoDetectado** visíveis.*

*Destaque visual: a chave **AlvoDetectado** do Blackboard, com uma seta indicando que ela é lida pelo nó Selector para escolher o ramo.*

Legenda sugerida: "A árvore decide; o Blackboard lembra."



O jogador ataca. Quem decide se o inimigo foi atingido, e quem decide o que fazer com isso?

Pense na diferença entre "detectar que um ataque acertou" e "saber o que fazer quando a vida chega a zero".

Combate simples: detecção de acerto + ApplyDamage

- Um combate simples não precisa de um sistema próprio de dano — só precisa identificar o alvo atingido
- `BP_Player` dispara um Trace (ou ativa um Overlap por um instante) durante uma ação de ataque
- Ao identificar um `BP_Enemy` atingido, chama `ApplyDamage` no `HealthComponent` do alvo — função já construída na Semana 8
- O `BP_Player` nunca precisa saber como o `HealthComponent` do inimigo guarda ou processa vida

DICA

Detectar o acerto (Trace/Overlap) e decidir o que fazer com o dano (`HealthComponent`) são dois problemas separados — o combate desta semana só resolve o primeiro, reaproveitando o segundo.

Combate simples: Unreal × Unity

Unreal

Line Trace/Overlap identifica o alvo;
chama `ApplyDamage` no
`HealthComponent` do alvo atingido

Unity

`Physics.Raycast` / `Physics.OverlapSphere`
identifica o alvo; chama um método público
de dano em um script de vida equivalente

Demonstração: combate simples

O professor dispara um Trace a partir de uma ação de ataque do `BP_Player`, identifica um `BP_Enemy` atingido e chama `ApplyDamage` no `HealthComponent` do inimigo.

Resultado esperado: a turma vê a vida do `BP_Enemy` sendo reduzida a cada acerto, sem nenhuma lógica de dano nova dentro do `HealthComponent`.

Desafio: comportamento autônomo adicional

Cada grupo propõe e implementa um comportamento autônomo adicional para seu `BP_Enemy` — alerta antes de perseguir, fuga ao ficar com pouca vida (reutilizando o `HealthComponent`), ou reação a uma interação do jogador — integrado ao NavMesh, à Behavior Tree e ao Blackboard já construídos.

ATENÇÃO

Não há indicação prévia de qual comportamento implementar. A escolha e a integração à Behavior Tree existente fazem parte do desafio.

Boas práticas

✓ BOAS PRÁTICAS

- Toda decisão do agente deve passar pela Behavior Tree, nunca por lógica solta no Event Graph do BP_Enemy
- Usar o Blackboard como único ponto de memória compartilhada do agente
- Reutilizar o HealthComponent já existente — nunca duplicar lógica de vida entre BP_Player e BP_Enemy

Playtest coletivo e Showcase — Encerramento do Módulo 3

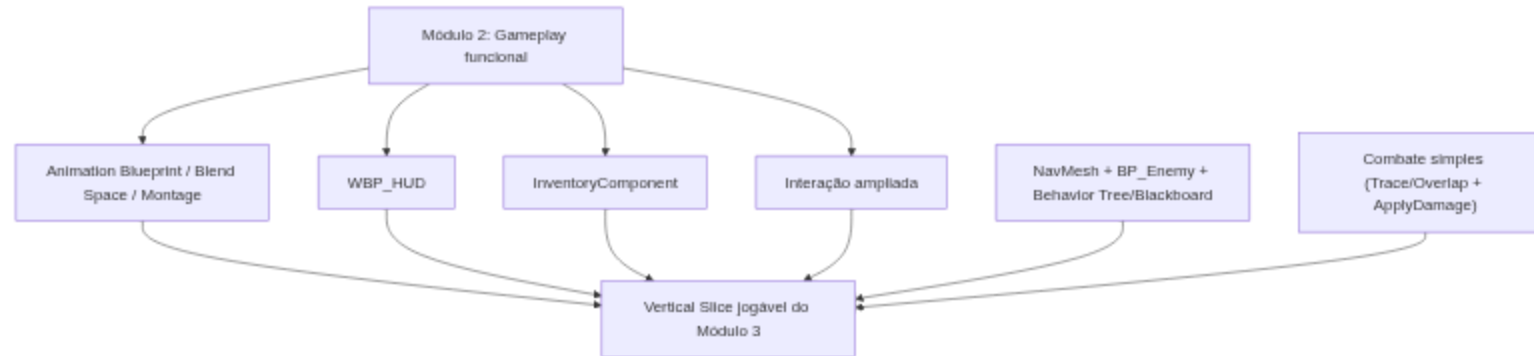
Cada grupo apresenta o Vertical Slice completo do Módulo 3 — animação, HUD, inventário, interação ampliada e IA integrados — em Showcase, seguido de Playtest coletivo conduzido pelos colegas.

OBJETIVOS

Critério de sucesso: Vertical Slice jogável do início ao fim, com `BP_Enemy` funcional, sem interromper nenhum sistema construído nos Módulos 1 e 2.

Tendências de Motores de Jogos

Vertical Slice do Módulo 3



IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Resumo da Semana 11

- NavMesh resolve deslocamento autônomo; Behavior Tree decide, Blackboard lembra
- `BP_Enemy` funcional, com patrulha, perseguição e um comportamento adicional proposto pelo grupo
- Combate simples funcional: Trace/Overlap do `BP_Player` chamando `ApplyDamage` no `HealthComponent` de `BP_Enemy`
- Vertical Slice do Módulo 3 apresentado em Showcase e avaliado em Playtest coletivo
- Unidade III — Resolver Problemas — encerrada

Checklist Final do Módulo 3

OBJETIVOS

- `HealthComponent` , Animation Blueprint com State Machine, Blend Space e Montage funcionais (Semana 8)
- `WBP_HUD` exibindo dados de jogo em tempo real (Semana 9)
- `InventoryComponent` e interação ampliada integrados (Semana 10)
- NavMesh gerado e `BP_Enemy` com patrulha, perseguição e comportamento adicional (Semana 11)
- Combate simples funcional entre `BP_Player` e `BP_Enemy` (Semana 11)
- Playtest coletivo e Showcase concluídos, com pendências registradas quando houver

Próximos passos



A Semana 12 abre a Unidade IV — Produzir como um Pequeno Estúdio, mudando a metodologia para Studio Based Learning com o professor como diretor técnico e autonomia alta. O foco passa a ser polimento técnico — materiais parametrizados (Material Instances) e Foliage — sobre o Vertical Slice já consolidado, sem substituir nenhum sistema construído até aqui.

Leitura recomendada: Behavior Trees in Unreal Engine (Epic Games Documentation).